

Agenda:-

- Max subarray of length = K . ✓
- Boundary printing
- Spiral matrix printing.
- Search an element in sorted matrix.

class starts at 9:05 PM.

Qw Given $arr[n]$, return max subarray sum of length = k .

examples: $arr[9] = [2, 4, 3, 7, 9, 8, 6, 5, 4]$

$k = 3$ → subarrays len = 3. Note: $k \leq arr.length$

subarrays:

s	e	sum.
0	2	$2 + 4 + 3 = 9$
1	3	$4 + 3 + 7 = 14$
2	4	19
3	5	24 → <u>Ans</u>
4	6	23
5	7	19
6	8	15

Brute force: Go to all the subarrays len = k
find sum of all subarrays
compare & return the ans.

Pseudo:

```
int maxSubarray(int[] arr, int k) {  
    int s = 0;    int ans = Integer.MIN_VALUE;  
    int e = k - 1;  
    while (e < n) {  
        int sum = 0;  
        for (int i = s; i <= e; i++) {  
            sum += arr[i];  
        }  
        ans = Math.max(ans, sum);  
        s++;  
        e++;  
    }  
    return ans;  
}
```

$O(n)$ ←

TC: $O(n^2)$
SC: $O(1)$

Approach:

tc: $O(n)$

$arr[9] = [2, 4, 3, 7, 9, 8, 6, 5, 4]$

$k=3$ → subarrays len=3.

subarray → $s \quad e$ → $sum(0,2) = arr[0] + arr[1] + arr[2]$

→ $1 \quad 3$ → $sum(0,3) = arr[1] + arr[2] + arr[3]$

$sum(s,e) \Rightarrow$ Prefix sum

$arr[9] = [2, 4, 3, 7, 9, 8, 6, 5, 4]$ $k=3$

$pf[9] = [2, 6, 9, 16, 25, 33, 39, 44, 48]$

s	e	$sum(s,e) = pf[e] - pf[s-1]$
0	2	$arr[0] + arr[1] + arr[2] \Rightarrow pf[2] = 9$
1	3	$sum(1,3) \rightarrow pf[3] - pf[0]$ $= 16 - 2 = 14$
2	4	$sum(2,4) \rightarrow pf[4] - pf[1]$ $= 25 - 6 = 19$
3	5	$sum(3,5) \rightarrow pf[5] - pf[2]$ $33 - 9 = 24$
$\vdots$	$\vdots$	$\vdots$
6	8	$sum(6,8) = pf[8] - pf[5]$ $= 48 - 33 = 15$

Pseudocode:

```
int maxSubarray(int[] arr, int k) {  
    int s = 0;  
    int e = k-1;  
    int ans = Integer.MIN-VALUE;  
    sc: O(n) ← int[] pf = prefixSum(arr); → Refer from pfx sum class  
    O(n) ← while( e < n) {  
        if (s == 0) {  
            sum = pf[e];  
        } else {  
            sum = pf[e] - pf[s-1];  
        }  
        ans = Math.max(ans, sum);  
        s++;  
        e++;  
    }  
    return ans;  
}
```

TC: $O(n)$

SC: $O(n)$

Approach 3:

TC: $O(n)$

SC: $O(1)$

$arr[9] = [\overset{0}{2} \overset{1}{4} \overset{2}{3} \overset{3}{7} \overset{4}{9} \overset{5}{8} \overset{6}{6} \overset{7}{5} \overset{8}{4}] \quad k=3$

(window) subarray no.	s	e	sum
1st	0	2	$sum = arr[0] + arr[1] + arr[2]$ $= 9. \text{ (alone)}$
2nd	1	3	$sum = \cancel{arr[0]} + arr[1] + arr[2]$ $\quad \quad \quad - \cancel{arr[0]} + arr[3]$ $sum = sum - arr[0] + arr[3]$ $\uparrow \quad \quad \quad \uparrow$ $s-1 \quad \quad \quad e$
3rd	2	4	$sum = sum - arr[1] + arr[4]$ $\uparrow \quad \quad \quad \uparrow$ $s-1 \quad \quad \quad e$
4th	3	5	$sum = sum - arr[2] + arr[5]$ $\uparrow \quad \quad \quad \uparrow$ $s-1 \quad \quad \quad e$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
	6	8	$sum = sum - arr[5] + arr[8]$ $\uparrow \quad \quad \quad \uparrow$ $s-1 \quad \quad \quad e$
	s	e	$sum = sum - arr[s-1] + arr[e]$

Pseudo code:

```
int maxSubarraySum(int[] arr, int k) {  
    // calculate sum for first subarray [0, k-1]  
    int sum = 0;  
    for (i = 0; i <= k-1; i++) {  
        sum = sum + arr[i];  
    }  
    int s = 1;  
    int e = k;  
    int ans = sum;  
    while (e < n) {  
        sum = sum - arr[s-1] + arr[e];  
        ans = Math.max(ans, sum);  
        s++;  
        e++;  
    }  
    return ans;  
}
```

Tc: $O(n)$

Sc: $O(1)$

Ques:

Boundary printing [clockwise]

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

→

1	2	3	4	5
10	15	20	25	
24	23	22	21	
16	11	6		

cheudish approach

1st row	→	1	2	3	4	5
last col	→	5	10	15	20	25
last row [rev]	→	25	24	23	22	21
1st col [rev]	→	21	16	11	6	1

Smart:

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

Green ÷ 1 2 3 4

Yellow ÷ 5 10 15 20

Blue ÷ 25 24 23 22

Red ÷ 21 16 11 6

Op

<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>
<u>10</u>	<u>15</u>	<u>20</u>	25	
24	23	22	21	
16	11	6		

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

$n=5$

Green:
(4 el)
 $n-1$

Starting pt

i	j	
0	0	$\rightarrow 1$
0	1	$\rightarrow 2$
0	2	$\rightarrow 3$
0	3	$\rightarrow 4$
0	4	$\rightarrow \text{stop}$

$i=0$ and $j=4$.

Yellow:
(4 el)
 $n-1$

i	j	
0	4	$\rightarrow 5$
1	4	$\rightarrow 10$
2	4	$\rightarrow 15$
3	4	$\rightarrow 20$
4	4	$\rightarrow \text{stop}$

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

$el = 25$ ($n \times n$)

Green Yellow Blue Red
 $n-1$ $n-1$ $n-1$ $n-1$

$$\boxed{4n-4}$$

Blue:
(4 el)
 $n-1$

$i=4$ and $j=4$.

i	j	
4	4	$\rightarrow 25$
4	3	$\rightarrow 24$
4	2	$\rightarrow 23$
4	1	$\rightarrow 22$
4	0	$\rightarrow \text{stop}$

Red:
(4 el)
 $n-1$

$i=4$ and $j=0$

i	j	
4	0	$\rightarrow 21$
3	0	$\rightarrow 16$
2	0	$\rightarrow 11$
1	0	$\rightarrow 6$
0	0	$\rightarrow \text{stop}$

Pseudocode:

```
void printBoundary ( int[][] ) {  
    Clockwise    mat
```

```
    int n = arr.length;
```

```
    int i = 0; → row no
```

```
    int j = 0; → col no
```

```
    // Print green color
```

```
    for ( times = 1; times <= n-1; times++ ) {
```

```
        print ( mat[i][j] );
```

```
        j++;
```

```
    }
```

```
    // Print yellow color
```

```
    for ( times = 1; times <= n-1; times++ ) {
```

```
        print ( mat[i][j] );
```

```
        i++;
```

```
    }
```

```
    // Print blue color
```

```
    for ( times = 1; times <= n-1; times++ ) {
```

```
        print ( mat[i][j] );
```

```
        j--;
```

```
    }
```

```
    // Print red color
```

```
    for ( times = 1; times <= n-1; times++ ) {
```

```
        print ( mat[i][j] );
```

```
        i--;
```

```
    }
```

```
}
```

TC: $O(4n-4) \approx O(n)$

SC: $O(1)$

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

Q. spiral matrix printing [clockwise]

Even length matrix.

	0	1	2	3	4	5
0	1	2	3	4	5	6
1	7	8	9	10	11	12
2	13	14	15	16	17	18
3	19	20	21	22	23	24
4	25	26	27	28	29	30
5	31	32	33	34	35	36

→

1	2	3	4	5	6	12	18
24	30	36	35	34	33	32	
31	25	19	13	7	8	9	
10	11	17	23			29	
28	27	26	20	14	15	16	
22	21						

Yellow, Green, Red.

Yellow: $i=0$ and $j=0$ (boundary print)

Green: $i=1$ and $j=1$ (boundary print)

Red: $i=2$ and $j=2$ (boundary print)

$n=6$

i	j	no of prints
0	0	5 (n-1)
+1	+1	-2
+1	1	3
	2	-2
		1
		-2
		-1 → stop

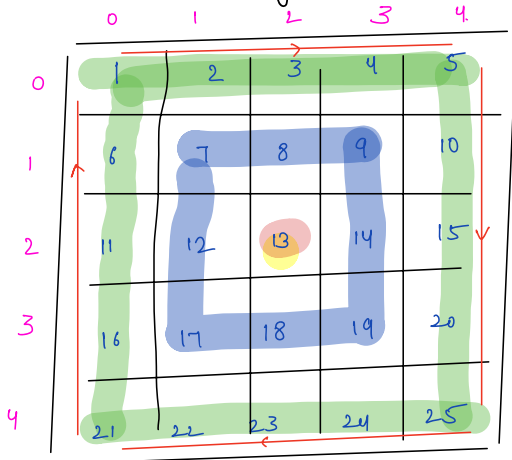
Color:
Yellow
Green.
Red

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36

odd length matrix



output

```

1  2  3  4  5  10  15  20
25 24 23 22 21  16  11
6   7   8   9  14  19  18
17  12  13

```

i j

0 0

+1 ↶

0 1 2 3 4 1

+1 ↶

2 2

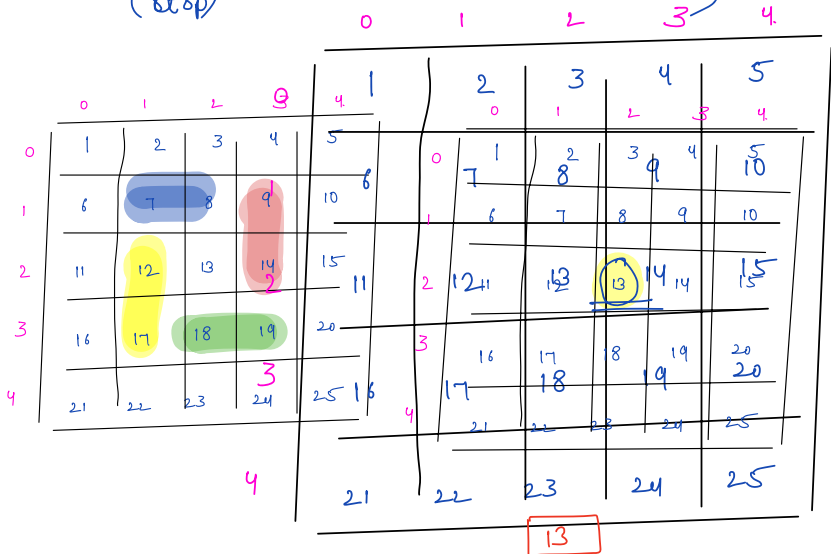
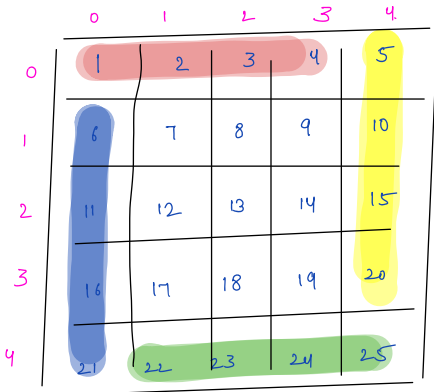
times color

4 Green

2 Blue

0 Red

edge case: times = 0;
1 el to print



illogical ← print 0 times

edge case: 0
↑
print it alone

```

void spiralPrinting(int [][] matrix) {
    int n = matrix.length;
    int i = 0;
    int j = 0;
    int k = n-1;
    while( k >= 0 ) {
        if( k == 0 ) {
            print(mat[i][j]);
            break;
        }

        // Print green color
        for( times = 1; times <= k; times++ ) {
            print(mat[i][j]);
            j++;
        }

        // Print yellow color
        for( times = 1; times <= k; times++ ) {
            print(mat[i][j]);
            i++;
        }

        // Print blue color
        for( times = 1; times <= k; times++ ) {
            print(mat[i][j]);
            j--;
        }

        // Print red color
        for( times = 1; times <= k; times++ ) {
            print(mat[i][j]);
            i--;
        }

        i++;
        j++;
        k = k-2;
    }
}

```

TC : $O(n^2)$
SC : $O(1)$

Ques Given a rowwise and colwise sorted matrix. find k .
 $mat[n][n]$ { $m \times n$ matrix } $\rightarrow$ easy work

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36

$k = 22 \rightarrow \{ \text{true} \}$

$k = 38 \rightarrow \{ \text{false} \}$

$k = 15 \rightarrow \{ \text{true} \}$

Brute force:

boolean findK(int[][] mat, int k)

int n = mat.length

for(i=0; i<n; i++) {

for(j=0; j<n; j++) {

if(mat[i][j] == k) {

return true;

}

| }

return false;

}

TC: $O(n^2)$

SC: $O(1)$

Index

	0	1	2	3	4	5
0	-10	-5	-2	2	4	7
1	-7	-4	-1	3	6	9
2	-2	3	5	7	11	14
3	2	6	8	11	14	<u>17</u>
4	7	11	12	15	19	20
5	10	14	18	20	24	29

$$k=8$$

8 is not present in first row.

last el of first row < 8 .
(7)

$$k=11.$$

1st row: NO, $7 < 11$.

2nd row: NO, $9 < 11$.

3rd row: $14 > 11$. [possibility]

$$k=10.$$

1st row $\rightarrow$ nothing $7 < 10$

2nd row $\rightarrow 9 < 10 \rightarrow$ nothing

3rd row: $14 > 10$ (Traverse)

4th row: $17 > 10$ (Traverse)

5th row: $20 > 10$ [Traverse]

6th row: $29 > 10$ [Traverse] $\rightarrow$ found my ans.

TC: $O(n^2)$
SC: $O(1)$

Row is sorted.

Idea3:

	0	1	2	3	4	5
0	-10	-5	-2	2	4	7
1	-7	-4	-1	3	6	9
2	-2	3	5	7	11	14
3	2	6	8	11	14	17
4	7	11	12	15	19	20
5	10	14	18	20	24	29

top right

k=12

i=0 j=5

$$\text{mat}[i][j] = 12$$

$$\text{mat}[i][j] > 12$$

$$\text{mat}[i][j] < 12 \quad \left[\begin{array}{l} k \text{ is not in} \\ \text{other row} \end{array} \right]$$

skip row, go to next row

→ i=1, j=5, k=12

$$\text{mat}[1][5] < 12. \quad \left[\text{skip row, go to next row} \right]$$

→ i=2, j=5, k=12

$$\text{mat}[2][5] > 12. \quad \left[\text{skip col, go to prev col} \right]$$

→ i=2, j=4, k=12

$$\text{mat}[2][4] < 12. \quad \left[\text{skip row, go to next row} \right]$$

→ i=3, j=4, k=12

$$\text{mat}[3][4] > 12 \quad \left[\text{skip col, prev col} \right]$$

→ i=3, j=3, k=12

$$\text{mat}[3][3] < 12 \quad \left\{ i++ \right\}$$

	0	1	2	3	4	5
0	-10	-5	-2	2	4	7
1	-7	-4	-1	3	6	9
2	-2	3	5	7	11	14
3	2	6	8	11	14	17
4	7	11	12	15	19	20
5	10	14	18	20	24	29

→ $i=4$ and $j=3$, $k=12$

$\text{mat}[4][3] > 12$ $\{j--\}$
15

→ $i=4$ and $j=2$, $k=12$

$\text{mat}[4][2]$
 $12 = 12$ $\{ \text{return true} \}$
 ~~ϕ~~

Pseudocode:

```
boolean matSearch(int[][] mat, int k) {
```

```
    int i = 0;
```

```
    int j = n-1;
```

```
    while (  $i < n$  &&  $j >= 0$  ) {
```

```
        if (  $\text{mat}[i][j] == k$  ) {  
            return true;
```

```
        } else if (  $\text{mat}[i][j] < k$  ) {  
            // skip row.  
            i++;
```

```
        } else {  
            // skip col  
            j--;
```

```
        }
```

```
    }
```

```
    return false;
```

```
}
```

TC: $O(n)$ → linear

SC: $O(1)$

Thankyou 😊

Doubts:

	0	1	2	3	4	5
0	-10	-5	-2	2	4	7
1	-7	-4	-1	3	6	9
2	-2	3	5	7	11	14
3	2	6	8	11	14	17
4	7	11	12	15	19	20
5	10	14	18	20	24	29

k=12

mal

mal

$i=0$ and $j=5 \{n-1\}$

$mat[0][5] < 12$

7 < 12 $i++$

	0	1	2	3	4	5
0	-10	-5	-2	2	4	7
1	-7	-4	-1	3	6	9
2	-2	3	5	7	11	14
3	2	6	8	11	14	17
4	7	11	12	15	19	20
5	10	14	18	20	24	29

k=12

2

$n=5$

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

Yellow:

i	j	
0	0	(1)
0	1	
0	2	
0	3	
0	4	→ <u>stop</u>

Green:

$i=0$ & $j=4$.

0	4	
1	4	
2	4	
3	4	
4	4	→ <u>stop</u>

for ($k=0$; $k<4$; $k++$)
 print(mat[i][j])
 } $j--$

Blue:

$i=4$ & $j=4$.

4	4	→
4	3	
4	2	
4	1	
4	0	→ <u>stop</u>

Red:

i	j
4	0
3	0
2	0
1	0
0	0

→ stop

$i=0, j=0$

0	0	1	2	3	4	5
0	1	2	3	4	5	6
1	7	8	9	10	11	12
2	13	14	15	16	17	18
3	19	20	21	22	23	24
4	25	26	27	28	29	30
5	31	32	33	34	35	36

$k=0$
 ϕ

```

alPrinting(
int n = matrix.length
int i = 0;
int j = 0;
int k = n-1;
while (k >= 0) {
    if (k == 0) {
        print(mat[i][j]);
        break;
    }
    // Print green color
    for (times = 1; times <= k; times++) {
        print(mat[i][j]);
        j++;
    }
    // Print yellow color
    for (times = 1; times <= k; times++) {
        print(mat[i][j]);
        i++;
    }
    // Print blue color
    for (times = 1; times <= k; times++) {
        print(mat[i][j]);
        j--;
    }
    // Print red color
    for (times = 1; times <= k; times++) {
        print(mat[i][j]);
        i--;
    }
    i++;
    j++;
    k = k-2;
}

```

$i=0, j=0$

→ only one element is left to be printed